LABORATORY MANUAL

DATA STRUCTURES AND ALGORITHMS

(Laboratory Component)

Department of Computer Science and Engineering
Sri Chandrasekharendra Saraswathi Viswa Mahavidyalaya (SCSVMV)

LIST OF EXPERIMENTS

Exp. No.	Unit	Title of the Experiment
1	I	Implementation of Basic Array Operations (Insertion, Deletion, Traversal)
2	I	Building a Dynamic Employee Record System using Structure Pointers
3	II	Implementation of Stack and Expression Conversion - Infix to Postfix
4	II	Postfix Expression Evaluation using Stack
5	II	Implementation of Simple Queue using Array
6	II	Implementation of Priority Queue using Unordered Array
7	III	Implementation of Singly Linked List (Insertion, Deletion, and Traversal)
8	III	Implementation of Circular Queue using Linked List
9	IV	Implementation and Traversal (In-order, Pre-order, Post-order) of a Binary Search Tree
10	V	Implementation of Graph Search Algorithms (BFS and DFS)
11	V	Implementation of Bubble, Selection and Heap Sorts
12	V	Implementation of Hashing with Collision Resolution using Linear Probing

UNIT I - INTRODUCTION

EXPERIMENT NO. 1

Implementation of Basic Array Operations (Insertion, Deletion, Traversal)

Aim:

To write a C program to perform basic operations on a one-dimensional array such as insertion of an element at a given position, deletion of an element from a given position, and traversal of the array.

Algorithm:

Step 1: Start the program and declare an array of maximum size MAX.

Step 2: Read the number of elements n and read n elements into the array.

Step 3: Display a menu with options: Insert, Delete, Traverse and Exit.

Step 4: For Insert: read the position and value. Shift all elements from the last index up to the given position one place to the right. Place the new value at the given position and increment n.

Step 5: For Delete: read the position to be deleted. Shift all elements after that position one place to the left and decrement n.

Step 6: For Traverse: display all the elements of the array from index 0 to n-1.

Step 7: Repeat steps 3 to 6 until the user chooses Exit.

Step 8: Stop the program.

Program (C Language):

```
#include <stdio.h>

#define MAX 100

void traverse(int arr[], int n) {
    int i;
    printf("Array elements: ");
    for (i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");
}

int insert(int arr[], int n, int pos, int value) {
    int i;
    if (n >= MAX) {
        printf("Array is full. Cannot insert.\n");
```

```
        return n;
    }
    if (pos < 0 || pos > n) {
        printf("Invalid position.\n");
        return n;
    }
    for (i = n; i > pos; i--)
        arr[i] = arr[i - 1];
    arr[pos] = value;
    return n + 1;
}

int deleteElement(int arr[], int n, int pos) {
    int i;
    if (pos < 0 || pos >= n) {
        printf("Invalid position.\n");
        return n;
    }
    for (i = pos; i < n - 1; i++)
        arr[i] = arr[i + 1];
    return n - 1;
}

int main() {
    int arr[MAX], n, choice, pos, value;

    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter %d elements: ", n);
    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    do {
        printf("\n--- Array Operations Menu ---\n");
        printf("1. Insert\n2. Delete\n3. Traverse\n4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter position (0-indexed) and value to insert: ");
                scanf("%d %d", &pos, &value);
                n = insert(arr, n, pos, value);
                traverse(arr, n);
                break;
```

```
            case 2:
                printf("Enter position (0-indexed) to delete: ");
                scanf("%d", &pos);
                n = deleteElement(arr, n, pos);
                traverse(arr, n);
                break;
            case 3:
                traverse(arr, n);
                break;
            case 4:
                printf("Exiting program.\n");
                break;
            default:
                printf("Invalid choice.\n");
        }
    } while (choice != 4);

    return 0;
}
```

Sample Output:

```
Enter number of elements: 5
Enter 5 elements: 10 20 30 40 50

--- Array Operations Menu ---
1. Insert
2. Delete
3. Traverse
4. Exit
Enter your choice: 1
Enter position (0-indexed) and value to insert: 2 25
Array elements: 10 20 25 30 40 50

--- Array Operations Menu ---
1. Insert
2. Delete
3. Traverse
4. Exit
Enter your choice: 2
Enter position (0-indexed) to delete: 1
Array elements: 10 25 30 40 50
```

```
--- Array Operations Menu ---
1. Insert
2. Delete
3. Traverse
4. Exit
Enter your choice: 3
Array elements: 10 25 30 40 50

--- Array Operations Menu ---
1. Insert
2. Delete
3. Traverse
4. Exit
Enter your choice: 4
Exiting program.
```

Result:

Thus, a C program was written and executed successfully to perform insertion, deletion, and traversal operations on an array.

EXPERIMENT NO. 2

Building a Dynamic Employee Record System using Structure Pointers

Aim:

To write a C program to create a dynamic employee record system using structures and pointers, which allows storing, displaying, and searching employee records at run time using dynamic memory allocation.

Algorithm:

Step 1: Start the program and define a structure Employee with members id, name, and salary.

Step 2: Read the number of employees n from the user.

Step 3: Dynamically allocate memory for n employee records using malloc() and store the base address in a structure pointer.

Step 4: Read the id, name and salary for each of the n employees using pointer notation (emp + i)->member.

Step 5: Display a menu with options: Display all employees, Search employee by ID, and Exit.

Step 6: For Display: traverse the dynamically allocated array and print details of every employee.

Step 7: For Search: read an ID from the user, linearly search the array for a match and print the details if found, otherwise print a not-found message.

Step 8: Repeat steps 5 to 7 until the user chooses Exit.

Step 9: Free the dynamically allocated memory using free() and stop the program.

Program (C Language):

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Employee {
    int id;
    char name[50];
    float salary;
};

int main() {
    int n, i, choice;
    struct Employee *emp;
```

```
    printf("Enter number of employees: ");
    scanf("%d", &n);

    emp = (struct Employee *) malloc(n * sizeof(struct Employee));
    if (emp == NULL) {
        printf("Memory allocation failed.\n");
        return 1;
    }

    for (i = 0; i < n; i++) {
        printf("\nEnter details for employee %d\n", i + 1);
        printf("ID: ");
        scanf("%d", &(emp + i)->id);
        printf("Name: ");
        scanf("%s", (emp + i)->name);
        printf("Salary: ");
        scanf("%f", &(emp + i)->salary);
    }

    do {
        printf("\n--- Employee Record Menu ---\n");
        printf("1. Display all employees\n2. Search employee by ID\n3.
Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        if (choice == 1) {
            printf("\n%-6s %-20s %-10s\n", "ID", "Name", "Salary");
            for (i = 0; i < n; i++)
                printf("%-6d %-20s %-10.2f\n", (emp + i)->id, (emp + i)-
>name, (emp + i)->salary);
        } else if (choice == 2) {
            int searchId, found = 0;
            printf("Enter ID to search: ");
            scanf("%d", &searchId);
            for (i = 0; i < n; i++) {
                if ((emp + i)->id == searchId) {
                    printf("Found: ID=%d Name=%s Salary=%.2f\n",
                           (emp + i)->id, (emp + i)->name, (emp + i)-
>salary);
                    found = 1;
                    break;
                }
            }
            if (!found)
```

```
                printf("Employee with ID %d not found.\n", searchId);
            }
        } while (choice != 3);

        free(emp);
        return 0;
}
```

Sample Output:

```
Enter number of employees: 3

Enter details for employee 1
ID: 101
Name: Arun
Salary: 45000

Enter details for employee 2
ID: 102
Name: Bala
Salary: 52000

Enter details for employee 3
ID: 103
Name: Chitra
Salary: 48000

--- Employee Record Menu ---
1. Display all employees
2. Search employee by ID
3. Exit
Enter your choice: 1

ID     Name                 Salary
101    Arun                 45000.00
102    Bala                 52000.00
103    Chitra               48000.00

--- Employee Record Menu ---
1. Display all employees
2. Search employee by ID
3. Exit
Enter your choice: 2
```

```
Enter ID to search: 102
Found: ID=102 Name=Bala Salary=52000.00

--- Employee Record Menu ---
1. Display all employees
2. Search employee by ID
3. Exit
Enter your choice: 3
Exiting program.
```

Result:

Thus, a C program was written and executed successfully to build a dynamic employee record system using structure pointers.

UNIT II - STACKS AND QUEUES

EXPERIMENT NO. 3

Implementation of Stack and Expression Conversion - Infix to Postfix

Aim:

To write a C program to implement a stack using arrays (PUSH, POP, PEEK operations) and to convert a given infix expression into its equivalent postfix expression using a stack.

Algorithm:

Step 1: Start the program and declare a character array stack[MAX] with top initialized to -1.

Step 2: PUSH(x): if top equals MAX-1 report Stack Overflow, otherwise increment top and store x at stack[top].

Step 3: POP(): if top equals -1 report Stack Underflow, otherwise return stack[top] and decrement top.

Step 4: PEEK(): return the element at stack[top] without removing it.

Step 5: Read the infix expression as a string.

Step 6: Scan the infix expression from left to right, one character at a time.

Step 7: If the character is an operand (alphanumeric), append it directly to the postfix output.

Step 8: If the character is '(', PUSH it onto the stack.

Step 9: If the character is ')', POP and append operators to the output until '(' is encountered, then discard the '('.

Step 10: If the character is an operator, POP and append operators with precedence greater than or equal to the current operator, then PUSH the current operator.

Step 11: After the entire expression is scanned, POP and append all remaining operators from the stack to the output.

Step 12: Display the postfix expression and stop the program.

Program (C Language):

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

#define MAX 100
```

```
char stack[MAX];
int top = -1;

void push(char c) {
    if (top == MAX - 1)
        printf("Stack Overflow\n");
    else
        stack[++top] = c;
}

char pop() {
    if (top == -1) {
        printf("Stack Underflow\n");
        return -1;
    }
    return stack[top--];
}

char peek() {
    if (top == -1)
        return -1;
    return stack[top];
}

int precedence(char c) {
    if (c == '^') return 3;
    if (c == '*' || c == '/') return 2;
    if (c == '+' || c == '-') return 1;
    return -1;
}

int isOperand(char c) {
    return isalnum(c);
}

void infixToPostfix(char infix[], char postfix[]) {
    int i, j = 0;
    for (i = 0; infix[i] != '\0'; i++) {
        char c = infix[i];
        if (isOperand(c)) {
            postfix[j++] = c;
        } else if (c == '(') {
            push(c);
        } else if (c == ')') {
            while (top != -1 && peek() != '('
```

```
                postfix[j++] = pop();
            pop(); /* remove '(' */
        } else {
            while (top != -1 && precedence(peek()) >= precedence(c))
                postfix[j++] = pop();
            push(c);
        }
    }
    while (top != -1)
        postfix[j++] = pop();
    postfix[j] = '\0';
}

int main() {
    char infix[MAX], postfix[MAX];
    printf("Enter an infix expression: ");
    scanf("%s", infix);
    infixToPostfix(infix, postfix);
    printf("Postfix expression: %s\n", postfix);
    return 0;
}
```

Sample Output:

```
Enter an infix expression: A+B*(C-D)/E
Postfix expression: ABCD-*E/+
```

Result:

Thus, a C program was written and executed successfully to implement a stack and convert an infix expression to its postfix form.

EXPERIMENT NO. 4

Postfix Expression Evaluation using Stack

Aim:

To write a C program to evaluate a given postfix expression using a stack.

Algorithm:

Step 1: Start the program and declare a stack of type float with top initialized to -1.

Step 2: Read the postfix expression as a string.

Step 3: Scan the postfix expression from left to right, one character at a time.

Step 4: If the scanned character is a digit (operand), convert it to a number and PUSH it onto the stack.

Step 5: If the scanned character is an operator, POP the top two elements from the stack; let the first popped value be op2 and the second be op1.

Step 6: Apply the operator on op1 and op2 (op1 operator op2) and PUSH the result back onto the stack.

Step 7: Repeat steps 3 to 6 until all the characters of the postfix expression are scanned.

Step 8: After the scan is complete, POP the final result from the stack; this is the value of the expression.

Step 9: Display the result and stop the program.

Program (C Language):

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
#include <math.h>

#define MAX 100

float stack[MAX];
int top = -1;

void push(float val) {
    if (top == MAX - 1)
        printf("Stack Overflow\n");
    else
        stack[++top] = val;
```

```
}

float pop() {
    if (top == -1) {
        printf("Stack Underflow\n");
        return 0;
    }
    return stack[top--];
}

float evaluatePostfix(char postfix[]) {
    int i;
    float op1, op2, result;

    for (i = 0; postfix[i] != '\0'; i++) {
        char c = postfix[i];

        if (isdigit(c)) {
            push(c - '0');
        } else if (isspace(c)) {
            continue;
        } else {
            op2 = pop();
            op1 = pop();
            switch (c) {
                case '+': result = op1 + op2; break;
                case '-': result = op1 - op2; break;
                case '*': result = op1 * op2; break;
                case '/': result = op1 / op2; break;
                case '^': result = pow(op1, op2); break;
                default:
                    printf("Invalid operator: %c\n", c);
                    result = 0;
            }
            push(result);
        }
    }
    return pop();
}

int main() {
    char postfix[MAX];
    printf("Enter a postfix expression (single-digit operands): ");
    scanf("%s", postfix);
    printf("Result = %.2f\n", evaluatePostfix(postfix));
```

```
    return 0;
}
```

Sample Output:

```
Enter a postfix expression (single-digit operands): 623*-8+
Result = 8.00
```

Result:

Thus, a C program was written and executed successfully to evaluate a postfix expression using a stack.

EXPERIMENT NO. 5

Implementation of Simple Queue using Array

Aim:

To write a C program to implement a simple (linear) queue using an array and to perform ENQUEUE, DEQUEUE, and display operations.

Algorithm:

Step 1: Start the program and declare an array queue[MAX] with front and rear initialized to -1.

Step 2: ENQUEUE(x): if rear equals MAX-1 report Queue Overflow. Otherwise, if front equals -1 set front to 0, increment rear and store x at queue[rear].

Step 3: DEQUEUE(): if front equals -1 or front is greater than rear, report Queue Underflow. Otherwise, display the element at queue[front], increment front, and if front exceeds rear reset both front and rear to -1.

Step 4: Display: if the queue is not empty, print all elements from index front to rear.

Step 5: Display a menu with options: Enqueue, Dequeue, Display and Exit.

Step 6: Repeat step 5 based on user choice until Exit is selected.

Step 7: Stop the program.

Program (C Language):

```
#include <stdio.h>

#define MAX 5

int queue[MAX];
int front = -1, rear = -1;

void enqueue(int val) {
    if (rear == MAX - 1) {
        printf("Queue Overflow\n");
        return;
    }
    if (front == -1)
        front = 0;
    queue[++rear] = val;
    printf("%d enqueued to queue.\n", val);
}
```

```
void dequeue() {
    if (front == -1 || front > rear) {
        printf("Queue Underflow\n");
        return;
    }
    printf("%d dequeued from queue.\n", queue[front]);
    front++;
    if (front > rear)
        front = rear = -1;
}

void display() {
    int i;
    if (front == -1 || front > rear) {
        printf("Queue is empty.\n");
        return;
    }
    printf("Queue elements: ");
    for (i = front; i <= rear; i++)
        printf("%d ", queue[i]);
    printf("\n");
}

int main() {
    int choice, val;
    do {
        printf("\n--- Simple Queue Menu ---\n");
        printf("1. Enqueue\n2. Dequeue\n3. Display\n4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter value to enqueue: ");
                scanf("%d", &val);
                enqueue(val);
                break;
            case 2:
                dequeue();
                break;
            case 3:
                display();
                break;
            case 4:
                printf("Exiting program.\n");
```

```
                break;
            default:
                printf("Invalid choice.\n");
        }
    } while (choice != 4);
    return 0;
}
```

Sample Output:

```
--- Simple Queue Menu ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter value to enqueue: 10
10 enqueued to queue.

--- Simple Queue Menu ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter value to enqueue: 20
20 enqueued to queue.

--- Simple Queue Menu ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter value to enqueue: 30
30 enqueued to queue.

--- Simple Queue Menu ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 3
```

```
Queue elements: 10 20 30

--- Simple Queue Menu ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 2
10 dequeued from queue.

--- Simple Queue Menu ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 3
Queue elements: 20 30

--- Simple Queue Menu ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 4
Exiting program.
```

Result:

Thus, a C program was written and executed successfully to implement a simple queue using an array.

EXPERIMENT NO. 6

Implementation of Priority Queue using Unordered Array

Aim:

To write a C program to implement a Priority Queue using an unordered array and to perform FindMin, Insert, and DeleteMin operations, where the element with the minimum value has the highest priority.

Algorithm:

Step 1: Start the program and declare an array pq[MAX] with count n initialized to 0.

Step 2: Insert(x): if the array is full report overflow, otherwise store x at pq[n] and increment n.

Step 3: FindMin(): scan the entire array pq[0..n-1] and find the index of the smallest element; return that index.

Step 4: DeleteMin(): call FindMin() to get the index of the minimum element, display it, then shift all subsequent elements one place to the left and decrement n.

Step 5: Display: print all the elements currently present in the array.

Step 6: Display a menu with options: Insert, FindMin, DeleteMin, Display and Exit.

Step 7: Repeat step 6 based on user choice until Exit is selected.

Step 8: Stop the program.

Program (C Language):

```
#include <stdio.h>

#define MAX 100

int pq[MAX];
int n = 0;

void insert(int val) {
    if (n == MAX) {
        printf("Priority Queue is full.\n");
        return;
    }
    pq[n++] = val;
    printf("%d inserted into priority queue.\n", val);
}

int findMinIndex() {
    int i, minIndex = 0;
```

```
    for (i = 1; i < n; i++)
        if (pq[i] < pq[minIndex])
            minIndex = i;
    return minIndex;
}

void findMin() {
    if (n == 0) {
        printf("Priority Queue is empty.\n");
        return;
    }
    printf("Minimum element (highest priority): %d\n", pq[findMinIndex()]);
}

void deleteMin() {
    int i, minIndex;
    if (n == 0) {
        printf("Priority Queue is empty. Cannot delete.\n");
        return;
    }
    minIndex = findMinIndex();
    printf("Deleted min element: %d\n", pq[minIndex]);
    for (i = minIndex; i < n - 1; i++)
        pq[i] = pq[i + 1];
    n--;
}

void display() {
    int i;
    if (n == 0) {
        printf("Priority Queue is empty.\n");
        return;
    }
    printf("Priority Queue elements: ");
    for (i = 0; i < n; i++)
        printf("%d ", pq[i]);
    printf("\n");
}

int main() {
    int choice, val;
    do {
        printf("\n--- Priority Queue Menu ---\n");
        printf("1. Insert\n2. FindMin\n3. DeleteMin\n4. Display\n5.
Exit\n");
```

```
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter value to insert: ");
                scanf("%d", &val);
                insert(val);
                break;
            case 2:
                findMin();
                break;
            case 3:
                deleteMin();
                break;
            case 4:
                display();
                break;
            case 5:
                printf("Exiting program.\n");
                break;
            default:
                printf("Invalid choice.\n");
        }
    } while (choice != 5);
    return 0;
}
```

Sample Output:

```
--- Priority Queue Menu ---
1. Insert
2. FindMin
3. DeleteMin
4. Display
5. Exit
Enter your choice: 1
Enter value to insert: 40
40 inserted into priority queue.

--- Priority Queue Menu ---
1. Insert
2. FindMin
```

```
3. DeleteMin
4. Display
5. Exit
Enter your choice: 1
Enter value to insert: 10
10 inserted into priority queue.

--- Priority Queue Menu ---
1. Insert
2. FindMin
3. DeleteMin
4. Display
5. Exit
Enter your choice: 1
Enter value to insert: 25
25 inserted into priority queue.

--- Priority Queue Menu ---
1. Insert
2. FindMin
3. DeleteMin
4. Display
5. Exit
Enter your choice: 4
Priority Queue elements: 40 10 25

--- Priority Queue Menu ---
1. Insert
2. FindMin
3. DeleteMin
4. Display
5. Exit
Enter your choice: 2
Minimum element (highest priority): 10

--- Priority Queue Menu ---
1. Insert
2. FindMin
3. DeleteMin
4. Display
5. Exit
Enter your choice: 3
Deleted min element: 10

--- Priority Queue Menu ---
```

```
1. Insert
2. FindMin
3. DeleteMin
4. Display
5. Exit
Enter your choice: 4
Priority Queue elements: 40 25

--- Priority Queue Menu ---
1. Insert
2. FindMin
3. DeleteMin
4. Display
5. Exit
Enter your choice: 5
Exiting program.
```

Result:

Thus, a C program was written and executed successfully to implement a priority queue using an unordered array.

UNIT III - LINKED LISTS

EXPERIMENT NO. 7

Implementation of Singly Linked List (Insertion, Deletion, and Traversal)

Aim:

To write a C program to implement a singly linked list and perform insertion (at the beginning, at a given position, and at the end), deletion (at the beginning, at a given position, and at the end), and traversal of the list.

Algorithm:

Step 1: Start the program and define a structure Node containing an integer data field and a pointer next. Initialize head to NULL.

Step 2: Insert at Beginning: create a new node, set its next to head, and make head point to the new node.

Step 3: Insert at End: create a new node with next as NULL. If the list is empty, make head point to it; otherwise traverse to the last node and link it to the new node.

Step 4: Insert at Position: traverse to the node just before the given position, link the new node's next to that node's next, and then link that node's next to the new node.

Step 5: Delete at Beginning: store head in a temporary pointer, move head to head->next, and free the temporary node.

Step 6: Delete at End: traverse to the second-last node, free the last node, and set the second-last node's next to NULL.

Step 7: Delete at Position: traverse to the node just before the given position, link it to the node after the one being deleted, and free the deleted node.

Step 8: Traverse: starting from head, print the data of each node and move to the next node until NULL is reached.

Step 9: Display a menu with all the above operations and repeat until the user chooses Exit.

Step 10: Stop the program.

Program (C Language):

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
```

```
    struct Node *next;
};

struct Node *head = NULL;

void insertBeginning(int val) {
    struct Node *newNode = (struct Node *) malloc(sizeof(struct Node));
    newNode->data = val;
    newNode->next = head;
    head = newNode;
}

void insertEnd(int val) {
    struct Node *newNode = (struct Node *) malloc(sizeof(struct Node));
    newNode->data = val;
    newNode->next = NULL;
    if (head == NULL) {
        head = newNode;
        return;
    }
    struct Node *temp = head;
    while (temp->next != NULL)
        temp = temp->next;
    temp->next = newNode;
}

void insertPosition(int val, int pos) {
    if (pos == 0) {
        insertBeginning(val);
        return;
    }
    struct Node *newNode = (struct Node *) malloc(sizeof(struct Node));
    newNode->data = val;
    struct Node *temp = head;
    int i;
    for (i = 0; i < pos - 1 && temp != NULL; i++)
        temp = temp->next;
    if (temp == NULL) {
        printf("Invalid position.\n");
        free(newNode);
        return;
    }
    newNode->next = temp->next;
    temp->next = newNode;
}
```

```
void deleteBeginning() {
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }
    struct Node *temp = head;
    head = head->next;
    free(temp);
}

void deleteEnd() {
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }
    if (head->next == NULL) {
        free(head);
        head = NULL;
        return;
    }
    struct Node *temp = head;
    while (temp->next->next != NULL)
        temp = temp->next;
    free(temp->next);
    temp->next = NULL;
}

void deletePosition(int pos) {
    if (pos == 0) {
        deleteBeginning();
        return;
    }
    struct Node *temp = head;
    int i;
    for (i = 0; i < pos - 1 && temp != NULL; i++)
        temp = temp->next;
    if (temp == NULL || temp->next == NULL) {
        printf("Invalid position.\n");
        return;
    }
    struct Node *toDelete = temp->next;
    temp->next = toDelete->next;
    free(toDelete);
}
```

```
void traverse() {
    struct Node *temp = head;
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }
    printf("Linked List: ");
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

int main() {
    int choice, val, pos;
    do {
        printf("\n--- Singly Linked List Menu ---\n");
        printf("1. Insert at Beginning\n2. Insert at Position\n3. Insert at
End\n");
        printf("4. Delete at Beginning\n5. Delete at Position\n6. Delete at
End\n");
        printf("7. Traverse\n8. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter value: ");
                scanf("%d", &val);
                insertBeginning(val);
                break;
            case 2:
                printf("Enter value and position (0-indexed): ");
                scanf("%d %d", &val, &pos);
                insertPosition(val, pos);
                break;
            case 3:
                printf("Enter value: ");
                scanf("%d", &val);
                insertEnd(val);
                break;
            case 4:
                deleteBeginning();
```

```
                break;
            case 5:
                printf("Enter position (0-indexed): ");
                scanf("%d", &pos);
                deletePosition(pos);
                break;
            case 6:
                deleteEnd();
                break;
            case 7:
                traverse();
                break;
            case 8:
                printf("Exiting program.\n");
                break;
            default:
                printf("Invalid choice.\n");
        }
    } while (choice != 8);
    return 0;
}
```

Sample Output:

```
--- Singly Linked List Menu ---
1. Insert at Beginning
2. Insert at Position
3. Insert at End
4. Delete at Beginning
5. Delete at Position
6. Delete at End
7. Traverse
8. Exit
Enter your choice: 3
Enter value: 10

Enter your choice: 3
Enter value: 20

Enter your choice: 3
Enter value: 30

Enter your choice: 2
```

```
Enter value and position (0-indexed): 15 1

Enter your choice: 7
Linked List: 10 -> 15 -> 20 -> 30 -> NULL

Enter your choice: 5
Enter position (0-indexed): 1

Enter your choice: 7
Linked List: 10 -> 20 -> 30 -> NULL

Enter your choice: 8
Exiting program.
```

Result:

Thus, a C program was written and executed successfully to implement insertion, deletion, and traversal operations on a singly linked list.

EXPERIMENT NO. 8

Implementation of Circular Queue using Linked List

Aim:

To write a C program to implement a circular queue using a singly linked list and to perform ENQUEUE, DEQUEUE, and display operations.

Algorithm:

Step 1: Start the program and define a structure Node containing an integer data field and a pointer next. Maintain only a rear pointer, where rear->next always points to the front node.

Step 2: Enqueue(x): create a new node. If the queue is empty, set the new node's next to itself and make rear point to it. Otherwise, link the new node's next to the current front (rear->next), link rear's next to the new node, and update rear to the new node.

Step 3: Dequeue(): if rear is NULL report Queue Underflow. Otherwise identify front as rear->next, display its data; if front is the same as rear (only one node) set rear to NULL, otherwise link rear->next to front->next; finally free the front node.

Step 4: Display: if the queue is empty print a message. Otherwise, starting from the front (rear->next), traverse the circular list once and print each node's data until the front node is reached again.

Step 5: Display a menu with options: Enqueue, Dequeue, Display and Exit and repeat until Exit is selected.

Step 6: Stop the program.

Program (C Language):

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

struct Node *rear = NULL; /* rear->next points to front (circular) */

void enqueue(int val) {
    struct Node *newNode = (struct Node *) malloc(sizeof(struct Node));
    newNode->data = val;
    if (rear == NULL) {
        newNode->next = newNode;
```

```
        rear = newNode;
    } else {
        newNode->next = rear->next;
        rear->next = newNode;
        rear = newNode;
    }
    printf("%d enqueued to circular queue.\n", val);
}

void dequeue() {
    if (rear == NULL) {
        printf("Queue Underflow\n");
        return;
    }
    struct Node *front = rear->next;
    printf("%d dequeued from circular queue.\n", front->data);
    if (front == rear) {
        rear = NULL;
    } else {
        rear->next = front->next;
    }
    free(front);
}

void display() {
    if (rear == NULL) {
        printf("Queue is empty.\n");
        return;
    }
    struct Node *temp = rear->next; /* front */
    printf("Circular Queue elements: ");
    do {
        printf("%d ", temp->data);
        temp = temp->next;
    } while (temp != rear->next);
    printf("\n");
}

int main() {
    int choice, val;
    do {
        printf("\n--- Circular Queue (Linked List) Menu ---\n");
        printf("1. Enqueue\n2. Dequeue\n3. Display\n4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
```

```
        switch (choice) {
            case 1:
                printf("Enter value to enqueue: ");
                scanf("%d", &val);
                enqueue(val);
                break;
            case 2:
                dequeue();
                break;
            case 3:
                display();
                break;
            case 4:
                printf("Exiting program.\n");
                break;
            default:
                printf("Invalid choice.\n");
        }
    } while (choice != 4);
    return 0;
}
```

Sample Output:

```
--- Circular Queue (Linked List) Menu ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter value to enqueue: 10
10 enqueued to circular queue.

Enter your choice: 1
Enter value to enqueue: 20
20 enqueued to circular queue.

Enter your choice: 1
Enter value to enqueue: 30
30 enqueued to circular queue.

Enter your choice: 3
```

```
Circular Queue elements: 10 20 30

Enter your choice: 2
10 dequeued from circular queue.

Enter your choice: 3
Circular Queue elements: 20 30

Enter your choice: 4
Exiting program.
```

Result:

Thus, a C program was written and executed successfully to implement a circular queue using a linked list.

UNIT IV - TREES

EXPERIMENT NO. 9

Implementation and Traversal (In-order, Pre-order, Post-order) of a Binary Search Tree

Aim:

To write a C program to implement a Binary Search Tree (BST) and to perform insertion and traversal of the tree using in-order, pre-order, post-order, and level-order techniques.

Algorithm:

Step 1: Start the program and define a structure Node containing data, and pointers left and right, both initialized to NULL for a new node.

Step 2: Insert(root, value): if root is NULL, create and return a new node with the given value. If value is less than root's data, recursively insert into the left subtree; if greater, recursively insert into the right subtree.

Step 3: In-order Traversal: recursively traverse the left subtree, visit the root node, then recursively traverse the right subtree.

Step 4: Pre-order Traversal: visit the root node, recursively traverse the left subtree, then recursively traverse the right subtree.

Step 5: Post-order Traversal: recursively traverse the left subtree, recursively traverse the right subtree, then visit the root node.

Step 6: Level-order Traversal: insert the root into a queue; while the queue is not empty, remove a node, display it, and insert its non-null left and right children into the queue.

Step 7: Read the number of nodes and their values, and build the BST by repeatedly calling Insert.

Step 8: Display a menu with the traversal options and an option to insert new nodes; repeat until Exit is selected.

Step 9: Stop the program.

Program (C Language):

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *left, *right;
};
```

```
struct Node *createNode(int val) {
    struct Node *newNode = (struct Node *) malloc(sizeof(struct Node));
    newNode->data = val;
    newNode->left = newNode->right = NULL;
    return newNode;
}

struct Node *insert(struct Node *root, int val) {
    if (root == NULL)
        return createNode(val);
    if (val < root->data)
        root->left = insert(root->left, val);
    else if (val > root->data)
        root->right = insert(root->right, val);
    return root;
}

void inorder(struct Node *root) {
    if (root != NULL) {
        inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);
    }
}

void preorder(struct Node *root) {
    if (root != NULL) {
        printf("%d ", root->data);
        preorder(root->left);
        preorder(root->right);
    }
}

void postorder(struct Node *root) {
    if (root != NULL) {
        postorder(root->left);
        postorder(root->right);
        printf("%d ", root->data);
    }
}

#define QMAX 100
void levelorder(struct Node *root) {
    struct Node *queue[QMAX];
    int front = 0, rear = 0;
```

```
    if (root == NULL) {
        printf("Tree is empty.\n");
        return;
    }
    queue[rear++] = root;
    while (front < rear) {
        struct Node *curr = queue[front++];
        printf("%d ", curr->data);
        if (curr->left != NULL)
            queue[rear++] = curr->left;
        if (curr->right != NULL)
            queue[rear++] = curr->right;
    }
}

int main() {
    struct Node *root = NULL;
    int n, i, val, choice;

    printf("Enter number of nodes to insert: ");
    scanf("%d", &n);
    printf("Enter %d values: ", n);
    for (i = 0; i < n; i++) {
        scanf("%d", &val);
        root = insert(root, val);
    }

    do {
        printf("\n--- BST Traversal Menu ---\n");
        printf("1. In-order\n2. Pre-order\n3. Post-order\n4. Level-
order\n5. Insert new node\n6. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("In-order Traversal: ");
                inorder(root);
                printf("\n");
                break;
            case 2:
                printf("Pre-order Traversal: ");
                preorder(root);
                printf("\n");
                break;
```

```
            case 3:
                printf("Post-order Traversal: ");
                postorder(root);
                printf("\n");
                break;
            case 4:
                printf("Level-order Traversal: ");
                levelorder(root);
                printf("\n");
                break;
            case 5:
                printf("Enter value to insert: ");
                scanf("%d", &val);
                root = insert(root, val);
                break;
            case 6:
                printf("Exiting program.\n");
                break;
            default:
                printf("Invalid choice.\n");
        }
    } while (choice != 6);
    return 0;
}
```

Sample Output:

```
Enter number of nodes to insert: 6
Enter 6 values: 50 30 70 20 40 60

--- BST Traversal Menu ---
1. In-order
2. Pre-order
3. Post-order
4. Level-order
5. Insert new node
6. Exit
Enter your choice: 1
In-order Traversal: 20 30 40 50 60 70

Enter your choice: 2
Pre-order Traversal: 50 30 20 40 70 60
```

```
Enter your choice: 3
Post-order Traversal: 20 40 30 60 70 50

Enter your choice: 4
Level-order Traversal: 50 30 70 20 40 60

Enter your choice: 6
Exiting program.
```

Result:

Thus, a C program was written and executed successfully to implement a Binary Search Tree and perform its traversals.

UNIT V - GRAPHS, SORTING AND HASHING

EXPERIMENT NO. 10

Implementation of Graph Search Algorithms (BFS and DFS)

Aim:

To write a C program to represent a graph using an adjacency matrix and to traverse it using the Breadth-First Search (BFS) and Depth-First Search (DFS) algorithms.

Algorithm:

Step 1: Start the program and read the number of vertices and edges of the graph. Build the adjacency matrix adj[n][n] by reading each edge (u, v) and setting adj[u][v] and adj[v][u] to 1.

Step 2: BFS(start): mark the start vertex as visited and insert it into a queue. While the queue is not empty, remove a vertex, display it, and for every unvisited adjacent vertex, mark it visited and insert it into the queue.

Step 3: DFS(start): mark the start vertex as visited and display it. Recursively visit each unvisited adjacent vertex of the current vertex in the same manner.

Step 4: Read the starting vertex for traversal.

Step 5: Display a menu with options BFS, DFS and Exit; repeat based on user choice.

Step 6: Stop the program.

Program (C Language):

```
#include <stdio.h>

#define MAX 20

int adj[MAX][MAX];
int visited[MAX];
int n;

void bfs(int start) {
    int queue[MAX], front = 0, rear = 0, i;
    for (i = 0; i < n; i++)
        visited[i] = 0;

    visited[start] = 1;
    queue[rear++] = start;

    printf("BFS Traversal: ");
```

```
    while (front < rear) {
        int curr = queue[front++];
        printf("%d ", curr);
        for (i = 0; i < n; i++) {
            if (adj[curr][i] == 1 && !visited[i]) {
                visited[i] = 1;
                queue[rear++] = i;
            }
        }
    }
    printf("\n");
}

void dfsUtil(int v) {
    int i;
    visited[v] = 1;
    printf("%d ", v);
    for (i = 0; i < n; i++) {
        if (adj[v][i] == 1 && !visited[i])
            dfsUtil(i);
    }
}

void dfs(int start) {
    int i;
    for (i = 0; i < n; i++)
        visited[i] = 0;
    printf("DFS Traversal: ");
    dfsUtil(start);
    printf("\n");
}

int main() {
    int edges, i, u, v, start, choice;

    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter number of edges: ");
    scanf("%d", &edges);

    for (i = 0; i < n; i++)
        for (v = 0; v < n; v++)
            adj[i][v] = 0;

    printf("Enter edges (u v) for an undirected graph:\n");
```

```
    for (i = 0; i < edges; i++) {
        scanf("%d %d", &u, &v);
        adj[u][v] = 1;
        adj[v][u] = 1;
    }

    printf("Enter starting vertex: ");
    scanf("%d", &start);

    do {
        printf("\n--- Graph Traversal Menu ---\n");
        printf("1. BFS\n2. DFS\n3. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                bfs(start);
                break;
            case 2:
                dfs(start);
                break;
            case 3:
                printf("Exiting program.\n");
                break;
            default:
                printf("Invalid choice.\n");
        }
    } while (choice != 3);
    return 0;
}
```

Sample Output:

```
Enter number of vertices: 5
Enter number of edges: 5
Enter edges (u v) for an undirected graph:
0 1
0 2
1 3
2 4
3 4
Enter starting vertex: 0
```

```
--- Graph Traversal Menu ---
1. BFS
2. DFS
3. Exit
Enter your choice: 1
BFS Traversal: 0 1 2 3 4

Enter your choice: 2
DFS Traversal: 0 1 3 4 2

Enter your choice: 3
Exiting program.
```

Result:

Thus, a C program was written and executed successfully to traverse a graph using BFS and DFS.

EXPERIMENT NO. 11

Implementation of Bubble, Selection and Heap Sorts

Aim:

To write a C program to sort a given list of elements using Bubble Sort, Selection Sort, and Heap Sort techniques.

Algorithm:

Step 1: Start the program and read n elements into an array.

Step 2: Bubble Sort: repeatedly compare each pair of adjacent elements in the array; if they are in the wrong order, swap them. Repeat this pass n-1 times so that after every pass the largest unsorted element bubbles to its correct position.

Step 3: Selection Sort: for each position i from 0 to n-2, find the index of the minimum element in the unsorted part of the array (from i to n-1) and swap it with the element at position i.

Step 4: Heap Sort - Build Max Heap: starting from the last non-leaf node down to the root, call heapify() so that the array satisfies the max-heap property (every parent is greater than or equal to its children).

Step 5: Heap Sort - Extract Elements: repeatedly swap the root (maximum element) with the last element of the heap, reduce the heap size by one, and call heapify() on the root to restore the max-heap property. Repeat until the heap size is 1.

Step 6: Display the sorted array after each sorting technique is applied.

Step 7: Display a menu with options Bubble Sort, Selection Sort, Heap Sort and Exit; repeat based on user choice.

Step 8: Stop the program.

Program (C Language):

```
#include <stdio.h>

void printArray(int arr[], int n) {
    int i;
    for (i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");
}

void bubbleSort(int arr[], int n) {
    int i, j, temp;
```

```
    for (i = 0; i < n - 1; i++) {
        for (j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

void selectionSort(int arr[], int n) {
    int i, j, minIndex, temp;
    for (i = 0; i < n - 1; i++) {
        minIndex = i;
        for (j = i + 1; j < n; j++) {
            if (arr[j] < arr[minIndex])
                minIndex = j;
        }
        temp = arr[minIndex];
        arr[minIndex] = arr[i];
        arr[i] = temp;
    }
}

void heapify(int arr[], int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;
    int temp;

    if (left < n && arr[left] > arr[largest])
        largest = left;
    if (right < n && arr[right] > arr[largest])
        largest = right;

    if (largest != i) {
        temp = arr[i];
        arr[i] = arr[largest];
        arr[largest] = temp;
        heapify(arr, n, largest);
    }
}

void heapSort(int arr[], int n) {
```

```
    int i, temp;
    for (i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    for (i = n - 1; i > 0; i--) {
        temp = arr[0];
        arr[0] = arr[i];
        arr[i] = temp;
        heapify(arr, i, 0);
    }
}

int main() {
    int arr[100], temp[100], n, i, choice;

    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter %d elements: ", n);
    for (i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    do {
        printf("\n--- Sorting Menu ---\n");
        printf("1. Bubble Sort\n2. Selection Sort\n3. Heap Sort\n4.
Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        for (i = 0; i < n; i++)
            temp[i] = arr[i];

        switch (choice) {
            case 1:
                bubbleSort(temp, n);
                printf("Sorted array (Bubble Sort): ");
                printArray(temp, n);
                break;
            case 2:
                selectionSort(temp, n);
                printf("Sorted array (Selection Sort): ");
                printArray(temp, n);
                break;
            case 3:
                heapSort(temp, n);
                printf("Sorted array (Heap Sort): ");
```

```
                printArray(temp, n);
                break;
            case 4:
                printf("Exiting program.\n");
                break;
            default:
                printf("Invalid choice.\n");
        }
    } while (choice != 4);
    return 0;
}
```

Sample Output:

```
Enter number of elements: 6
Enter 6 elements: 45 12 78 3 89 21

--- Sorting Menu ---
1. Bubble Sort
2. Selection Sort
3. Heap Sort
4. Exit
Enter your choice: 1
Sorted array (Bubble Sort): 3 12 21 45 78 89

Enter your choice: 2
Sorted array (Selection Sort): 3 12 21 45 78 89

Enter your choice: 3
Sorted array (Heap Sort): 3 12 21 45 78 89

Enter your choice: 4
Exiting program.
```

Result:

Thus, a C program was written and executed successfully to sort an array using Bubble Sort, Selection Sort, and Heap Sort.

EXPERIMENT NO. 12

Implementation of Hashing with Collision Resolution using Linear Probing

Aim:

To write a C program to implement a hash table using the division hash function and to resolve collisions using the linear probing technique, and to perform insertion, search, and display operations.

Algorithm:

Step 1: Start the program and declare a hash table of fixed SIZE along with a parallel array isOccupied[SIZE] initialized to 0.

Step 2: hashFunction(key): compute and return key % SIZE as the home index for the given key.

Step 3: Insert(key): compute index = hashFunction(key). While the slot at index is occupied (and does not already hold the same key), move to the next index using (index + 1) % SIZE, and report a collision. Once an empty slot is found, store the key there and mark it occupied.

Step 4: Search(key): compute index = hashFunction(key). While the slot at index is occupied, check if it holds the key; if found, report the index. Otherwise move to the next index using linear probing until the key is found or all slots have been checked.

Step 5: Display: print the index and the key stored (if any) for every slot of the hash table.

Step 6: Display a menu with options Insert, Search, Display and Exit; repeat based on user choice.

Step 7: Stop the program.

Program (C Language):

```
#include <stdio.h>

#define SIZE 10

int hashTable[SIZE];
int isOccupied[SIZE] = {0};

int hashFunction(int key) {
    return key % SIZE;
}

void insert(int key) {
    int index = hashFunction(key);
    int start = index;
    int count = 0;
```

```
    while (isOccupied[index] == 1) {
        if (hashTable[index] == key) {
            printf("Key %d already exists.\n", key);
            return;
        }
        index = (index + 1) % SIZE;
        count++;
        if (count == SIZE) {
            printf("Hash Table is full. Cannot insert %d\n", key);
            return;
        }
    }
    hashTable[index] = key;
    isOccupied[index] = 1;
    if (index != start)
        printf("Collision occurred for key %d. ", key);
    printf("Key %d inserted at index %d\n", key, index);
}

void search(int key) {
    int index = hashFunction(key);
    int start = index;
    int count = 0;

    while (isOccupied[index] == 1) {
        if (hashTable[index] == key) {
            printf("Key %d found at index %d\n", key, index);
            return;
        }
        index = (index + 1) % SIZE;
        count++;
        if (count == SIZE)
            break;
    }
    printf("Key %d not found in hash table.\n", key);
}

void display() {
    int i;
    printf("\nHash Table:\n");
    for (i = 0; i < SIZE; i++) {
        if (isOccupied[i])
            printf("Index %d: %d\n", i, hashTable[i]);
        else
            printf("Index %d: --\n", i);
```

```
    }
}

int main() {
    int choice, key;
    do {
        printf("\n--- Hashing (Linear Probing) Menu ---\n");
        printf("1. Insert\n2. Search\n3. Display\n4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter key to insert: ");
                scanf("%d", &key);
                insert(key);
                break;
            case 2:
                printf("Enter key to search: ");
                scanf("%d", &key);
                search(key);
                break;
            case 3:
                display();
                break;
            case 4:
                printf("Exiting program.\n");
                break;
            default:
                printf("Invalid choice.\n");
        }
    } while (choice != 4);
    return 0;
}
```

Sample Output:

```
--- Hashing (Linear Probing) Menu ---
1. Insert
2. Search
3. Display
4. Exit
Enter your choice: 1
```

```
Enter key to insert: 25
Key 25 inserted at index 5

Enter your choice: 1
Enter key to insert: 35
Collision occurred for key 35. Key 35 inserted at index 6

Enter your choice: 1
Enter key to insert: 15
Collision occurred for key 15. Key 15 inserted at index 7

Enter your choice: 3

Hash Table:
Index 0: --
Index 1: --
Index 2: --
Index 3: --
Index 4: --
Index 5: 25
Index 6: 35
Index 7: 15
Index 8: --
Index 9: --

Enter your choice: 2
Enter key to search: 35
Key 35 found at index 6

Enter your choice: 4
Exiting program.
```

Result:

Thus, a C program was written and executed successfully to implement hashing with collision resolution using linear probing.